

DENSITY BASED SPATIAL CLUSTERING OF APPLICATIONS WITH NOISE

Rachit Vargas (MBA | Engineer)

rachit.vargas@ulead.ac.cr

(+506) 8474-3543

Introducción

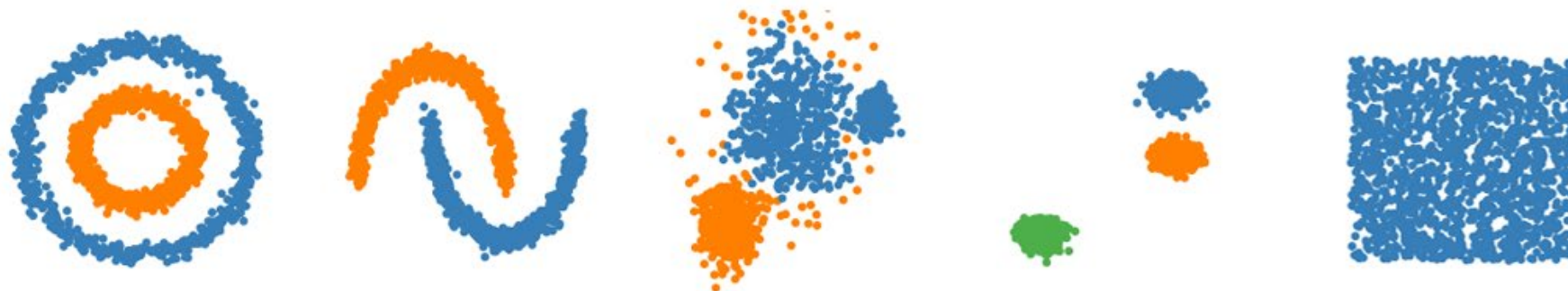
- DBSCAN es utilizado para identificar grupos de datos (clusters) a partir de la **densidad** de las observaciones en el espacio. A diferencia de métodos como *K-Means*, DBSCAN **no requiere conocer previamente el número de clusters** y es capaz de detectar clusters, además de identificar automáticamente los datos atípicos.
- El principio fundamental de DBSCAN consiste en que los puntos pertenecientes a una misma región densa forman un mismo cluster, mientras que los puntos aislados o con baja densidad son clasificados como ruido.
- Para ello, el algoritmo utiliza dos hiperparámetros: ϵ (**epsilon**), que define el radio de vecindad alrededor de un punto, y **MinPts**, que establece el número mínimo de vecinos necesarios para considerar que un punto pertenece a una región de alta densidad.

Introducción

- La intuición de DBSCAN se basa en la idea de que los datos tienden a organizarse de forma natural en regiones con alta densidad de observaciones.
- El algoritmo asume que los puntos que se encuentran próximos entre sí y poseen suficientes vecinos pertenecen al mismo grupo, mientras que las zonas de baja densidad actúan como una separación natural entre diferentes clusters.
- De esta manera, los grupos se forman mediante la conexión de regiones densas, permitiendo identificar estructuras de cualquier forma geométrica sin necesidad de definir previamente el número de clusters.
- Los puntos que no pertenecen a ninguna región suficientemente densa se consideran ruido (outliers) y se excluyen del proceso de agrupamiento.

DBSCAN VS KMEANS

DBSCAN



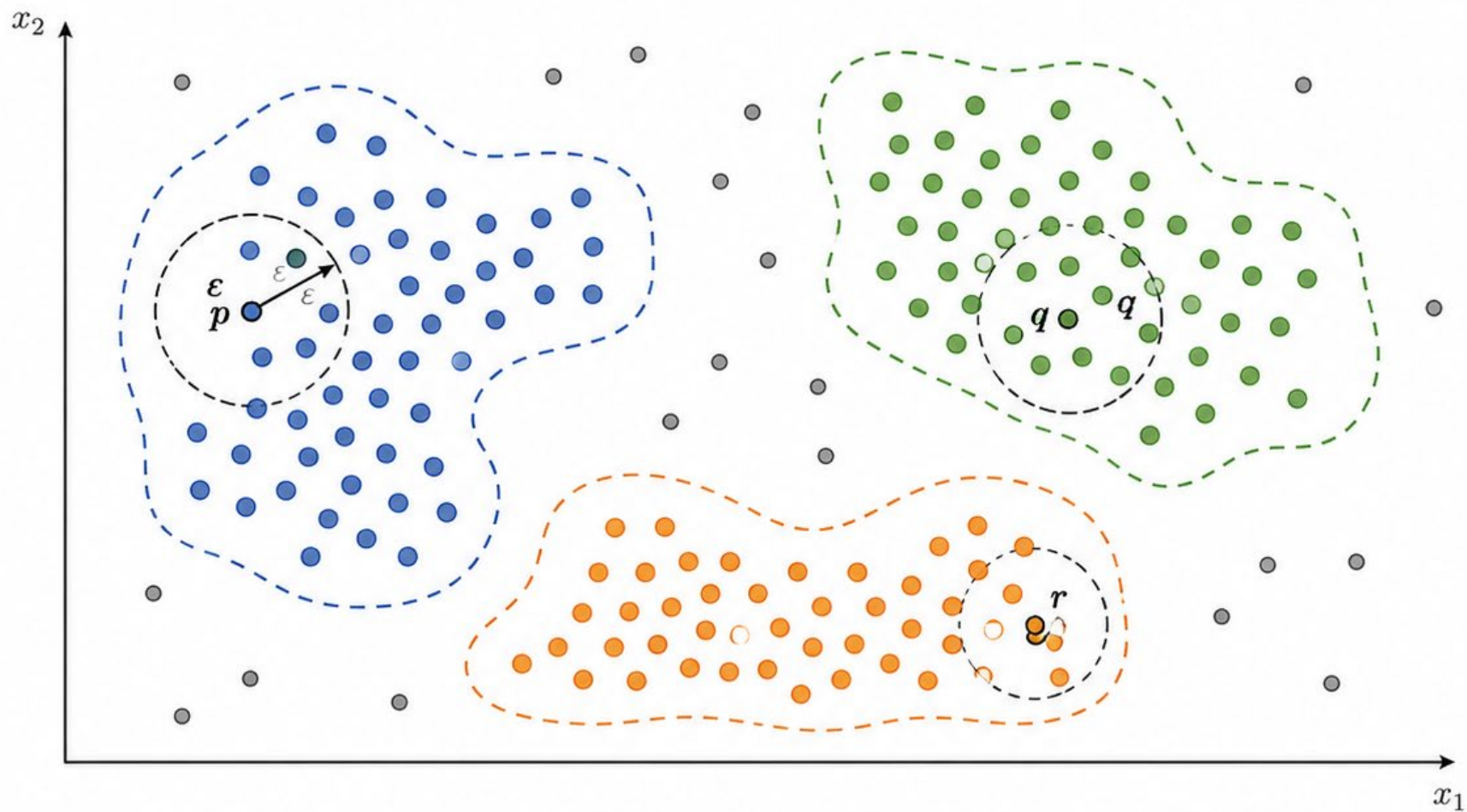
k-means



Sea $D \subseteq \mathbb{R}^d$ un conjunto de datos. DBSCAN agrupa puntos basándose en la densidad.

Dos parámetros controlan el algoritmo:

- $\varepsilon > 0$: radio del vecindario.
- $MinPts \in \mathbb{N}$: número mínimo de puntos para formar una región densa.

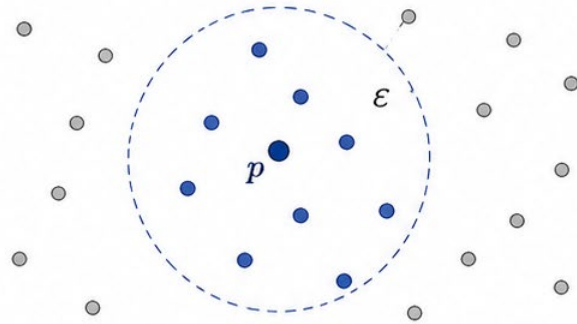


Tipos de puntos en DBSCAN

Sea $D \subseteq \mathbb{R}^d$ un conjunto de datos, $p \in D$ y $N_\epsilon(p) = \{x \in D : d(x, p) \leq \epsilon\}$ el ϵ -vecindario de p .
Un parámetro $MinPts \in \mathbb{N}$ determina la densidad mínima.

1. Punto central (core point)

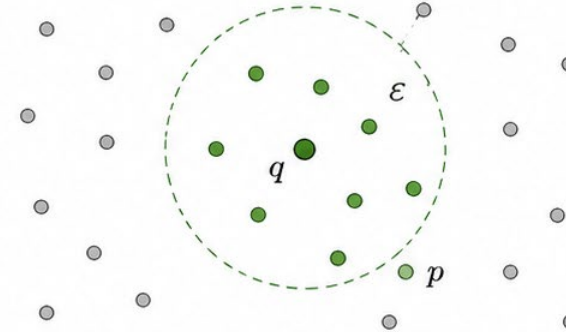
p es punto central si $|N_\epsilon(p)| \geq MinPts$.



El ϵ -vecindario de p contiene al menos $MinPts$ puntos.

2. Punto de frontera (border point)

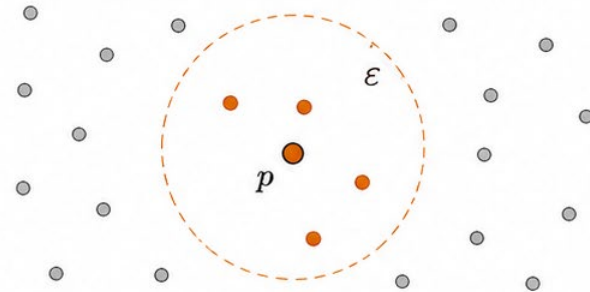
p no es central y existe q central tal que $p \in N_\epsilon(q)$.



p está en el vecindario de un punto central, pero su propio vecindario tiene menos de $MinPts$ puntos.

3. Punto de ruido (noise point)

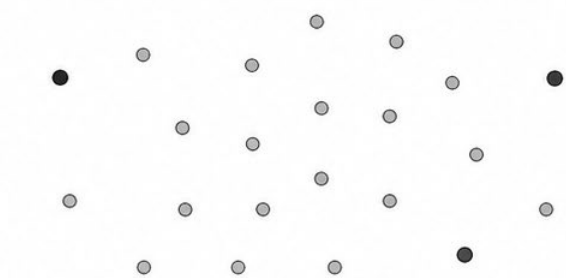
p no es central y no existe q central tal que $p \in N_\epsilon(q)$.



El ϵ -vecindario de p no contiene suficientes puntos y p no está en el vecindario de ningún punto central.

4. Ruido (outlier)

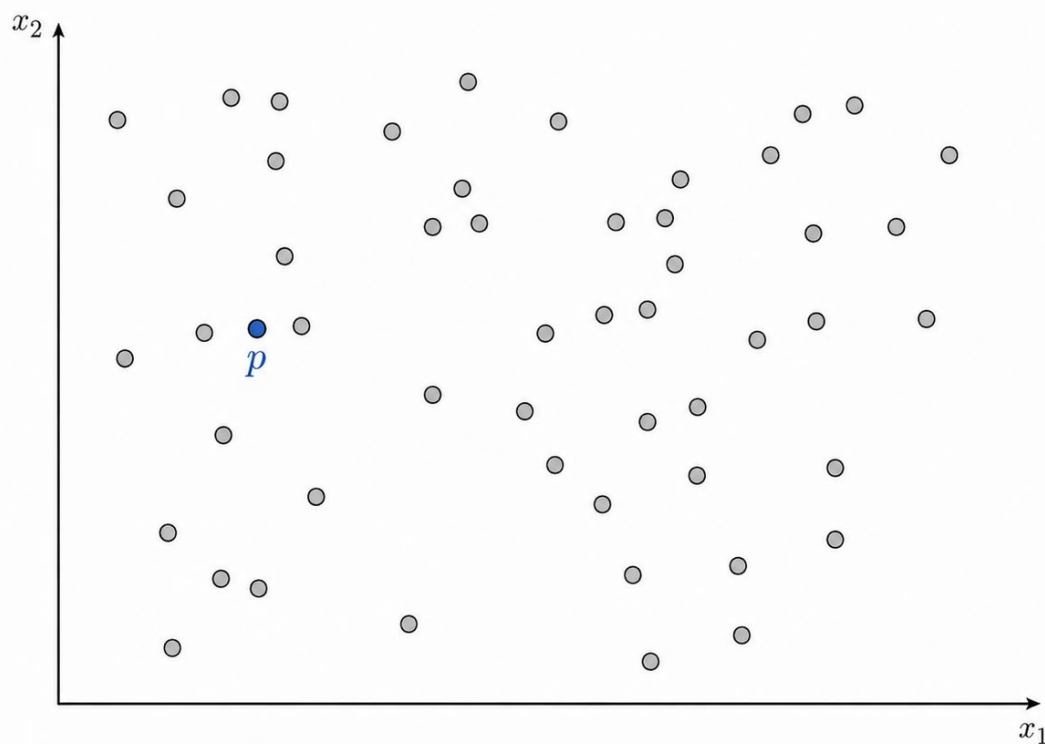
Puntos de ruido que no pertenecen a ningún cluster.



No son alcanzables desde ningún punto central.

1 Seleccionar un punto inicial.

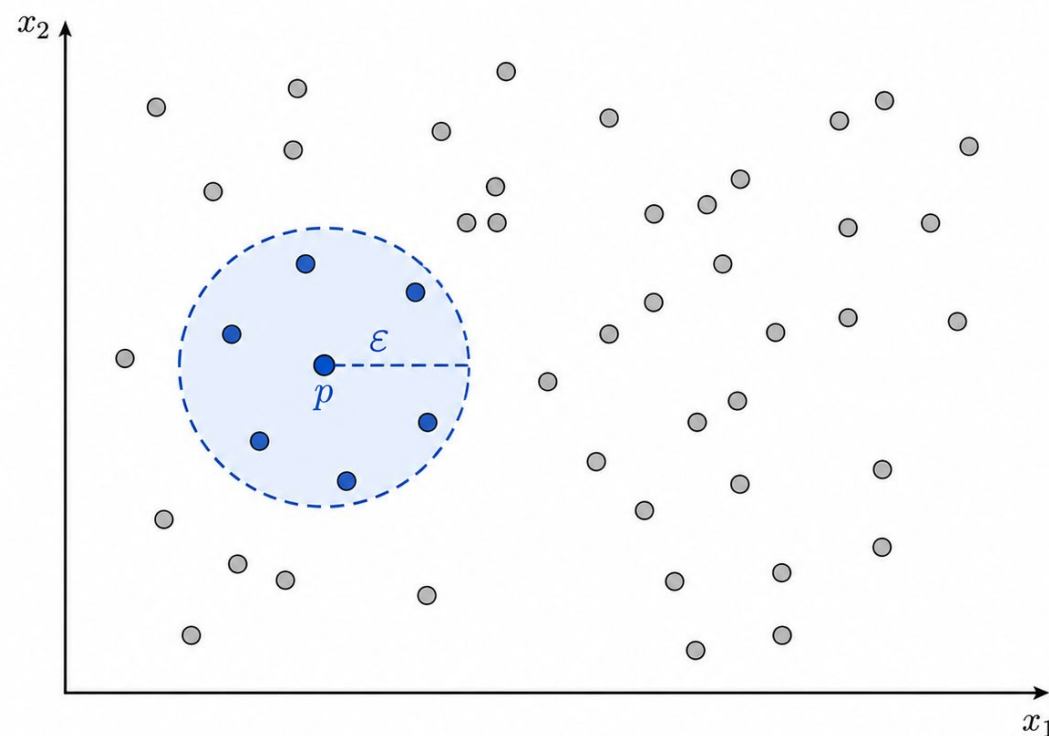
Elegir un punto p del conjunto de datos que aún no haya sido visitado.



- Punto seleccionado (p)
- Puntos no visitados

2 Construir el ε -vecindario del punto seleccionado.

Determinar todos los puntos que se encuentran a una distancia $\leq \varepsilon$ del punto p .



- | | |
|------------------------------|--|
| ● Punto seleccionado (p) | ● Puntos en el ε -vecindario de p |
| ● Puntos no visitados | ○ ε -vecindario (radio ε) |

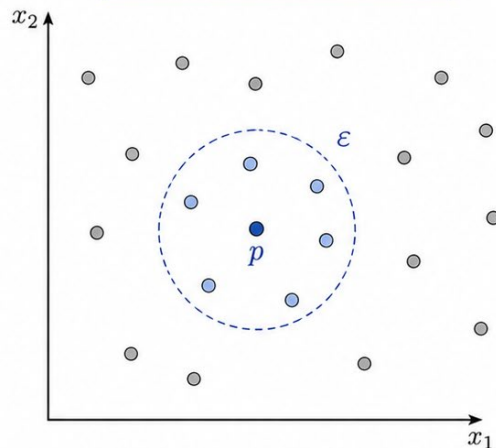
3 Clasificar el punto p (según $|N_\epsilon(p)|$ y $MinPts$).

Contar el número de puntos en el ϵ -vecindario de p (incluyendo p).

Si $|N_\epsilon(p)| \geq MinPts \Rightarrow p$ es punto central (*core point*).

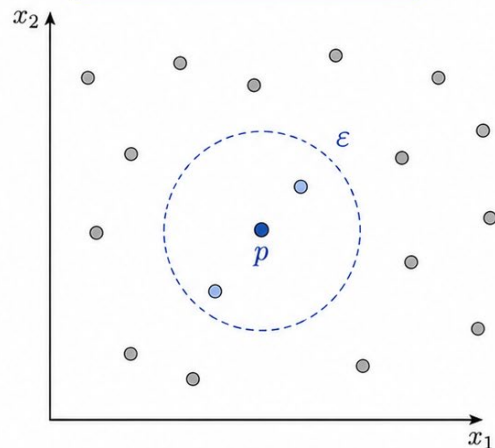
Si $|N_\epsilon(p)| < MinPts \Rightarrow p$ no es punto central.

Caso 1: $|N_\epsilon(p)| \geq MinPts$
 p es punto central.



Ejemplo: $|N_\epsilon(p)| = 9 \geq MinPts$

Caso 2: $|N_\epsilon(p)| < MinPts$
 p no es punto central.



Ejemplo: $|N_\epsilon(p)| = 3 < MinPts$

● Punto seleccionado (p)

● Puntos en el ϵ -vecindario de p

● Puntos no visitados

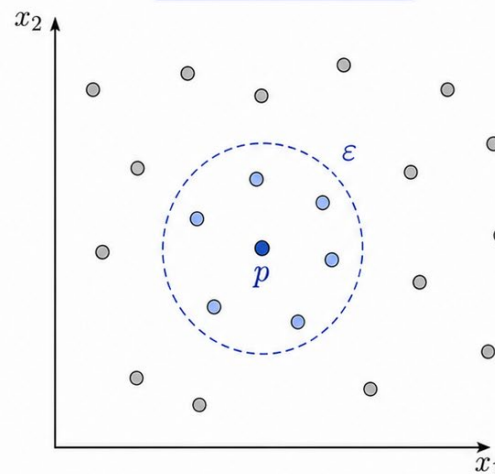
○ ϵ -vecindario de p (radio ϵ)

4 Si p es punto central, crear un nuevo cluster y agregar p al cluster.

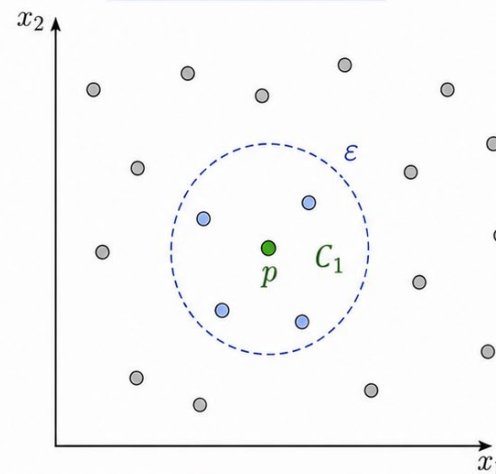
Asignar un nuevo identificador de cluster C_1 y marcar p como parte de C_1 .

(Este paso solo se ejecuta si $|N_\epsilon(p)| \geq MinPts$.)

Antes de este paso



Después de este paso



Se crea el nuevo cluster C_1
y p se agrega a C_1 .

● Punto seleccionado (p)

● Puntos en el ϵ -vecindario de p

● Puntos no visitados

● p agregado al nuevo cluster C_1

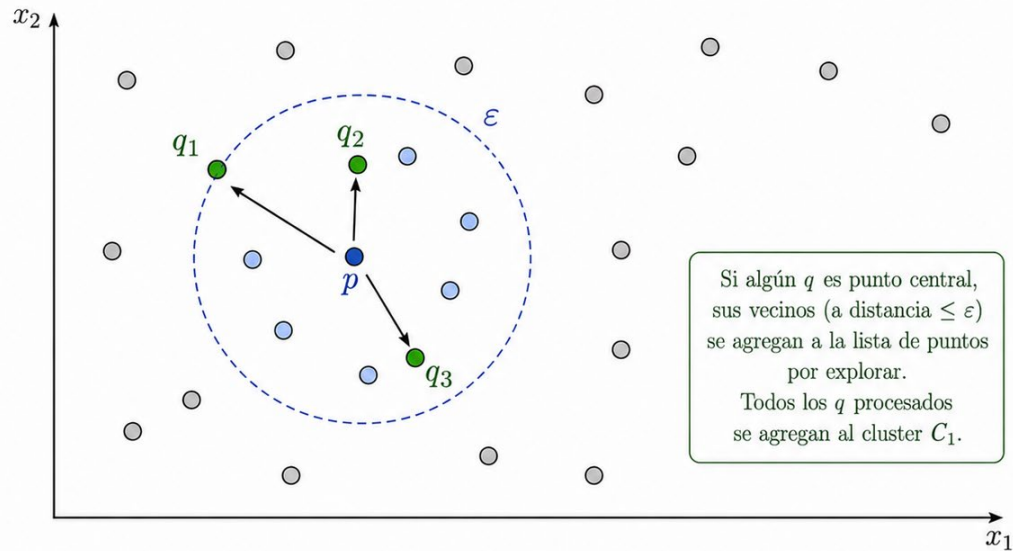
○ ϵ -vecindario de p (radio ϵ)

5

Explorar los puntos del ε -vecindario de p .

Para cada punto q en el ε -vecindario de p que aún no haya sido visitado:

- Si q es punto central ($|N_\varepsilon(q)| \geq \text{MinPts}$), agregar todos los puntos de $N_\varepsilon(q)$ que aún no estén en el cluster a la lista de puntos por explorar.
- Agregar q al cluster actual C_1 .



- | | |
|---|---|
| ● Punto seleccionado (p) | ● Puntos no visitados |
| ● Puntos en el ε -vecindario de p | ○ ε -vecindario de p (radio ε) |
| ● Puntos agregados al cluster C_1 | |

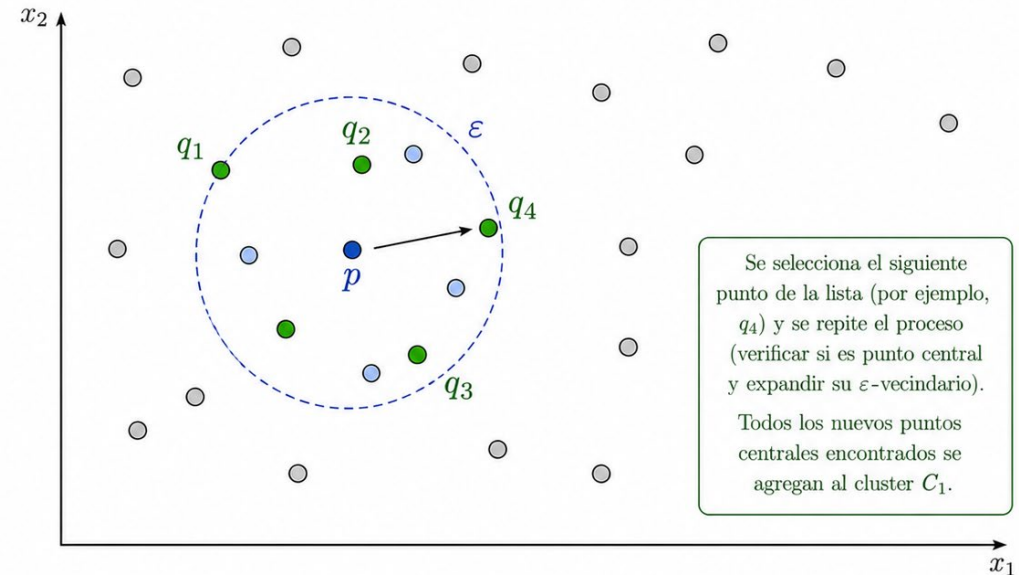
6

Repetir la exploración desde los nuevos puntos agregados.

Tomar el siguiente punto de la lista de puntos por explorar y repetir el proceso (verificar si es punto central y, en caso afirmativo, expandir su ε -vecindario).



Continuar hasta que la lista de puntos por explorar esté vacía.



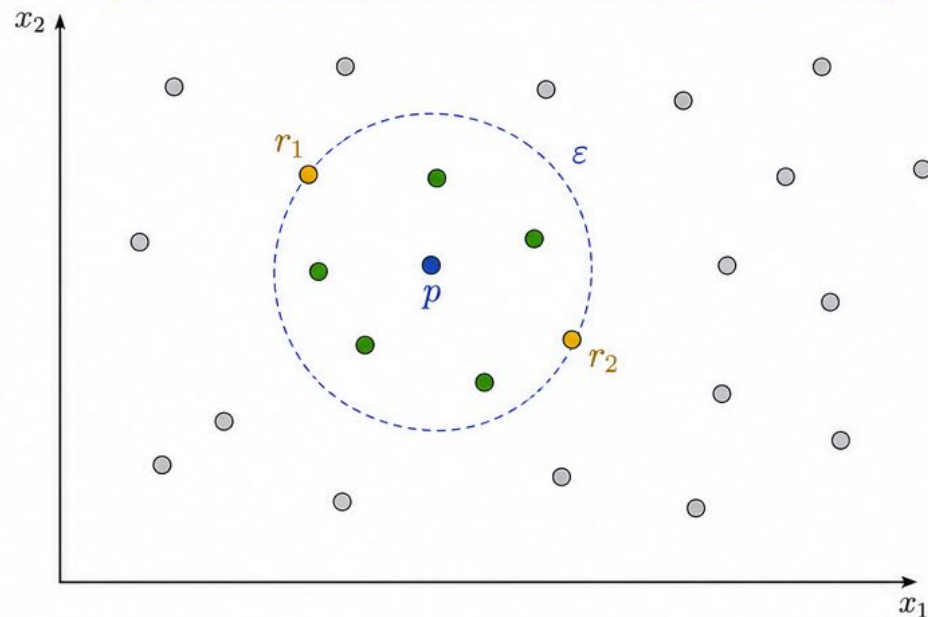
- | | |
|---|--|
| ● Punto seleccionado actualmente (p) | ● Puntos no visitados |
| ● Puntos en el ε -vecindario del punto actual | ○ ε -vecindario (radio ε) |
| ● Puntos ya agregados al cluster C_1 | |

7

Clasificar los puntos del ε -vecindario que no son puntos centrales.

Para cada punto r en el ε -vecindario de un punto central p que aún no haya sido visitado:

Si r no es punto central ($|N_\varepsilon(r)| < MinPts$) y aún no ha sido asignado a ningún cluster, entonces r es un **punto frontera** y se agrega al cluster actual C_1 .



- Punto central actual (p)
- Puntos ya agregados al cluster C_1 (centrales)
- Puntos frontera agregados a C_1
- Puntos no visitados
- ε -vecindario del punto actual

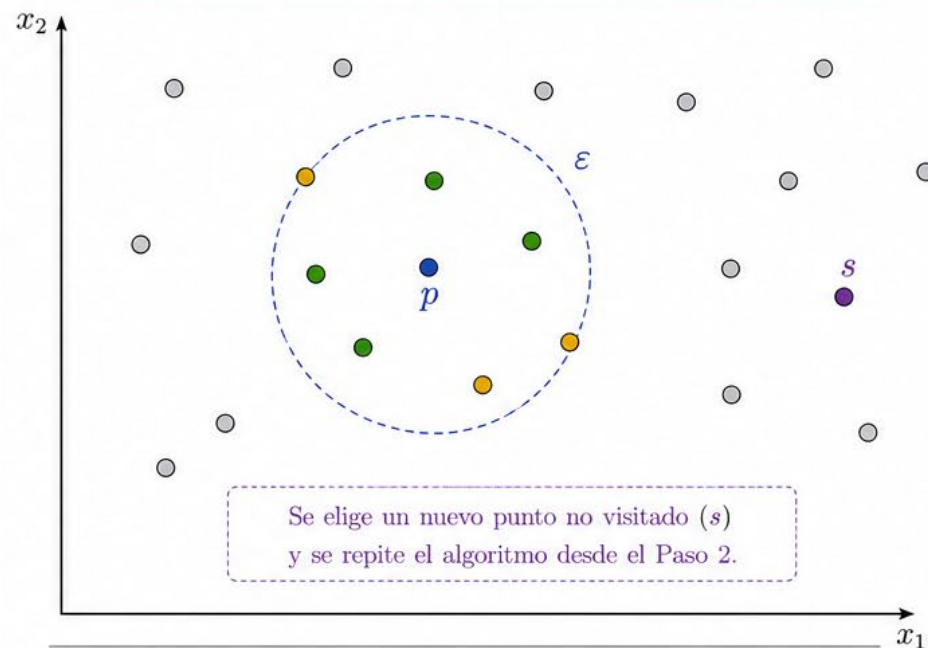
Los puntos frontera también pertenecen al cluster, pero no se expande desde ellos.

8

Cuando no queden puntos por explorar en el cluster actual.

Si la lista de puntos por explorar está vacía, el cluster actual C_1 queda finalizado.

Seleccionar el siguiente punto no visitado en el conjunto de datos y volver al **Paso 2** para verificar si es un nuevo punto central e iniciar un nuevo cluster.



Se elige un nuevo punto no visitado (s) y se repite el algoritmo desde el Paso 2.

- Puntos del cluster C_1 (centrales)
- Puntos del cluster C_1 (centrales)
- Puntos del cluster C_1 (frontera)
- Nuevo punto seleccionado (s)
- Puntos no visitados

El algoritmo continúa hasta que todos los puntos hayan sido visitados.

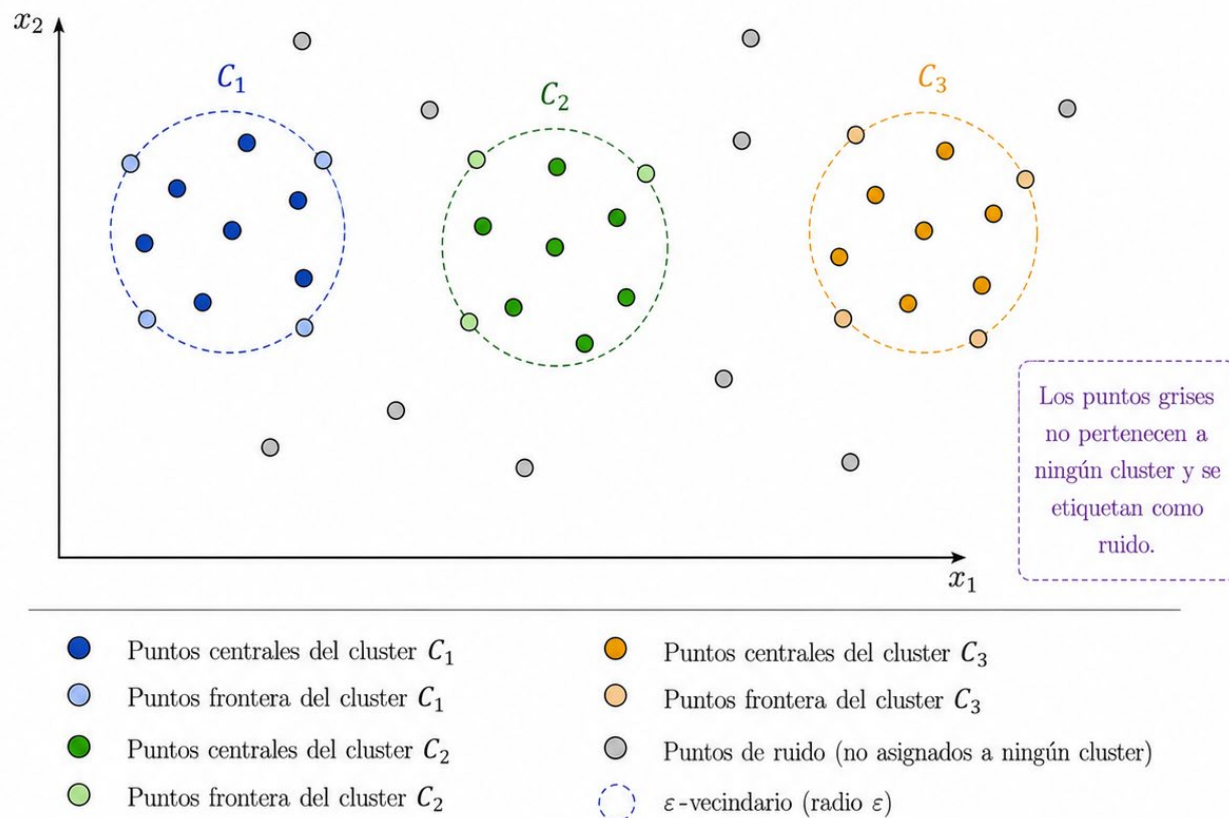
9

Asignar puntos de ruido y finalizar.

Después de procesar todos los puntos:

- Cualquier punto que no haya sido asignado a ningún cluster se etiqueta como ruido.

El algoritmo termina: todos los puntos están clasificados.

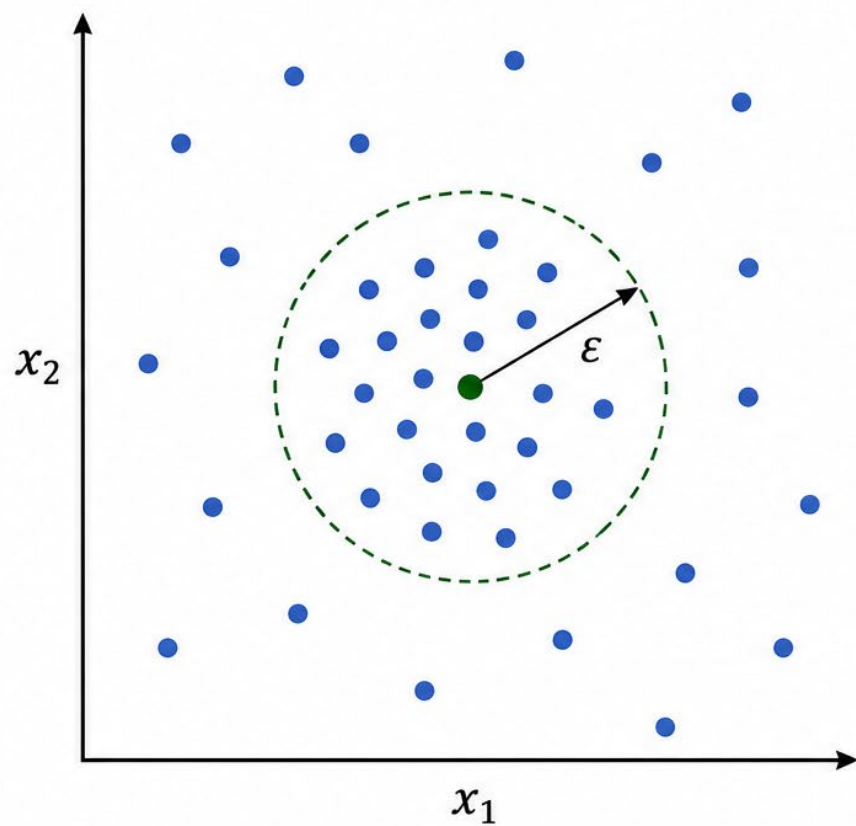


Densidad

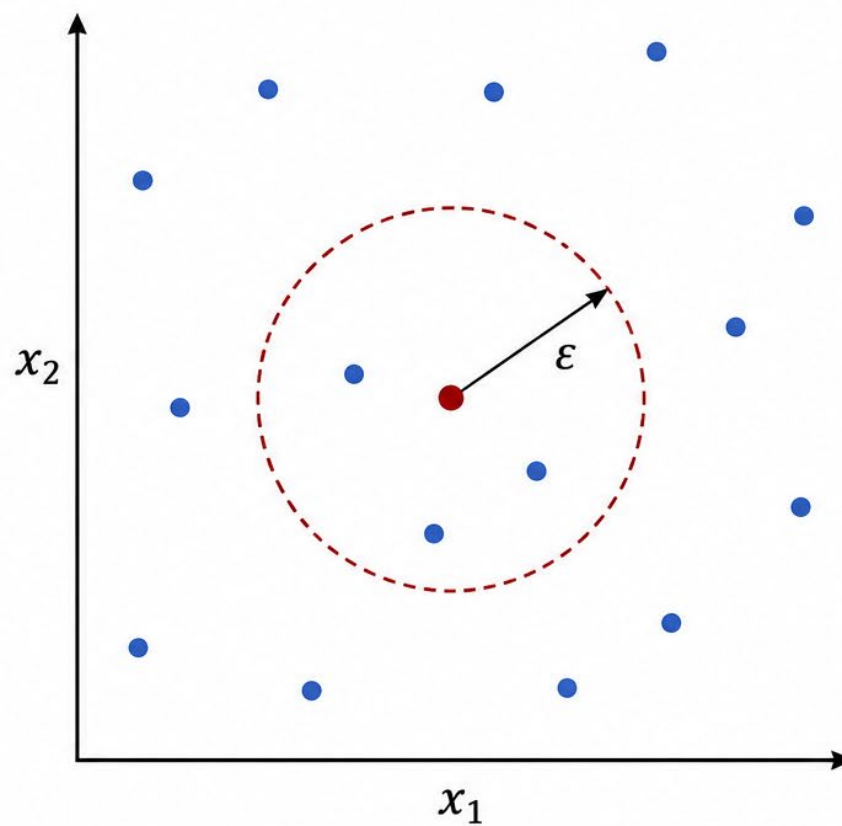
- Una región se considera **densa** cuando un punto posee un número suficiente de vecinos ubicados a una distancia menor o igual que un radio predefinido, denominado ϵ (**epsilon**).
- En contraste, las regiones donde existen pocos puntos cercanos presentan una baja densidad y, por tanto, es menos probable que formen parte de un cluster.
- De esta manera, el algoritmo no identifica grupos por su forma geométrica, sino por la distribución y concentración local de los datos. La relación entre la **distancia** y el **número de vecinos** es el fundamento del algoritmo.
- Si dos puntos se encuentran dentro del radio ϵ y la cantidad de observaciones cercanas alcanza el valor mínimo establecido por **MinPts**, dichos puntos pertenecen a una misma región densa.
- Esta propiedad permite establecer una **conectividad por densidad**, en la cual un cluster se expande mediante la unión de regiones densas adyacentes.

Concepto de Densidad

Región densa

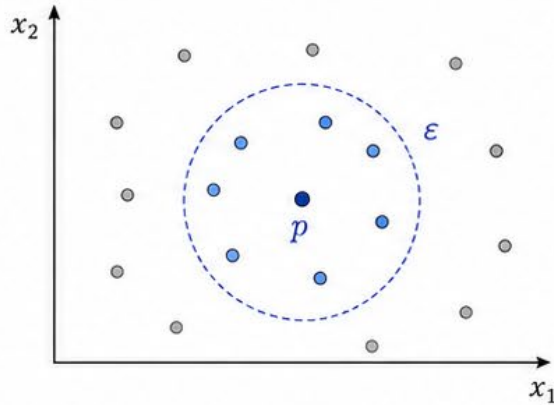


Región no densa

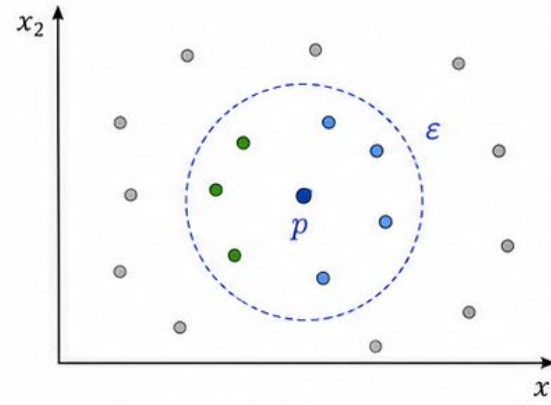


DBSCAN: expansión iterativa de un cluster

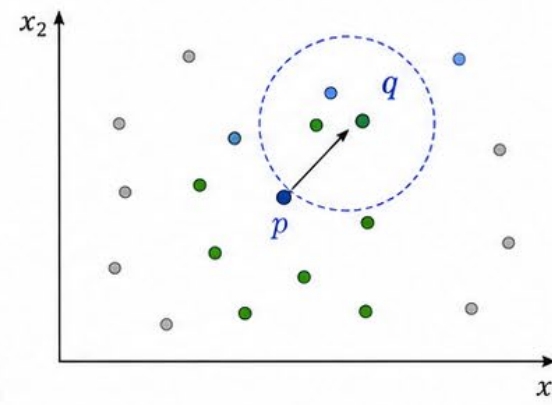
1. Seleccionar un punto inicial (p) y obtener su ε -vecindario



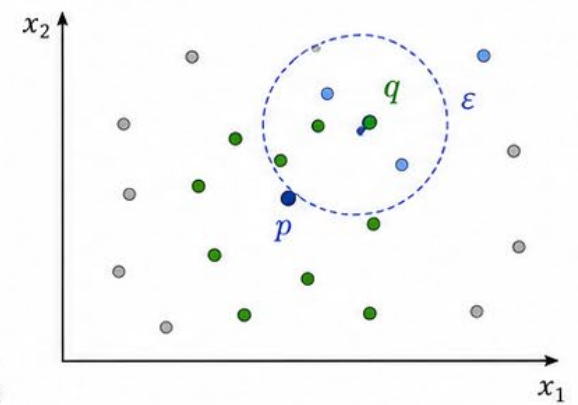
2. p es punto central $\rightarrow$ agregar vecinos al cluster



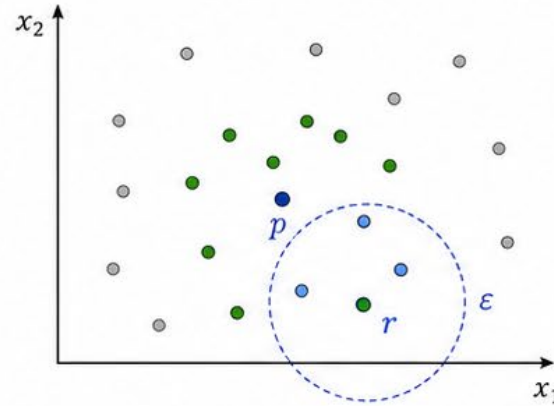
3. Tomar un nuevo punto del conjunto por explorar (q) y expandir



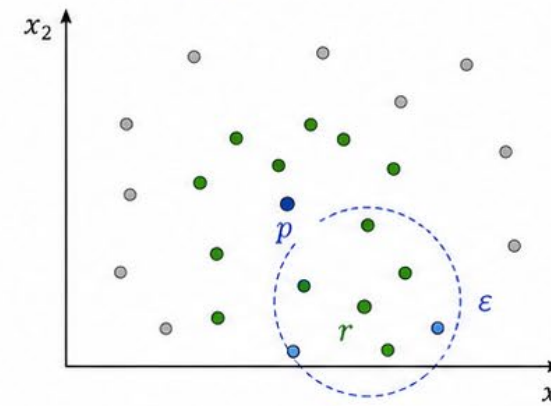
4. q es punto central $\rightarrow$ agregar sus vecinos



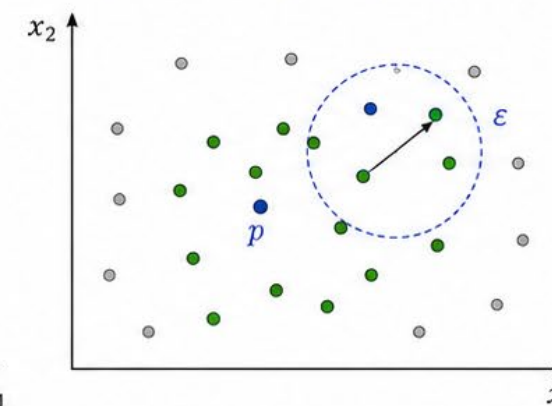
5. Tomar siguiente punto (r) y expandir



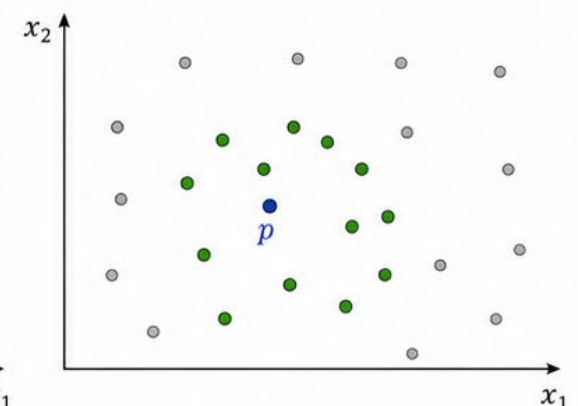
6. r es punto central $\rightarrow$ agregar sus vecinos



7. Repetir el proceso con los nuevos puntos del conjunto por explorar



8. No quedan puntos por explorar $\rightarrow$ cluster finalizado



● Punto inicial / punto central actual

● Vecinos en el ε -vecindario

● Puntos agregados al cluster

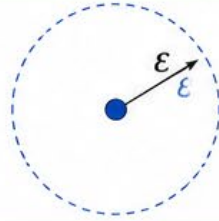
● Puntos no visitados (aún sin clasificar)

○ ε -vecindario (radio ε)

DBSCAN: LOS DOS HIPERPARÁMETROS

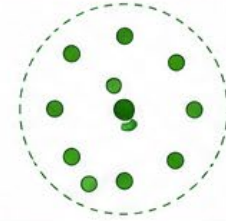
ϵ (Epsilon): radio de búsqueda.

Define el tamaño del vecindario alrededor de cada punto.



MinPts: cantidad mínima de vecinos.

Número mínimo de puntos (incluyendo el punto central) dentro del radio ϵ para que un punto sea considerado punto central.



Cómo afectan los resultados

ϵ

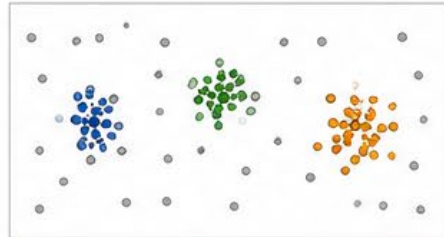
(Epsilon)

Controla el tamaño de la vecindad.

ϵ pequeño $\rightarrow$ vecindad pequeña.

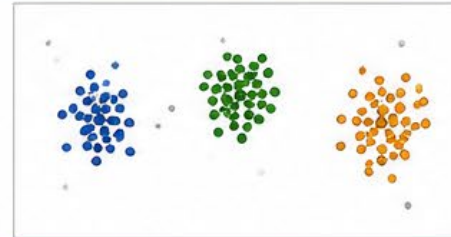
ϵ grande $\rightarrow$ vecindad grande.

ϵ pequeño



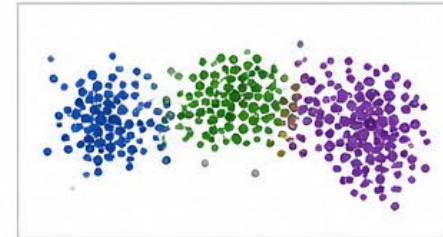
Mucho ruido. Se detectan muchos clusters pequeños o ninguno.

ϵ medio



Buen valor. Se identifican los clusters correctos.

ϵ grande



Vecindad muy grande. Los clusters se fusionan entre sí.

- Cluster 1
- Cluster 2
- Cluster 3
- Clusters fusionados
- Ruido (outliers)

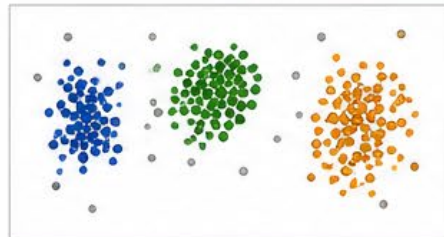
MinPts

Controla la densidad mínima requerida.

MinPts pequeño $\rightarrow$ más fácil ser punto central.

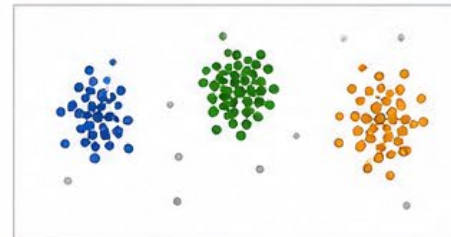
MinPts grande $\rightarrow$ se requiere más densidad.

MinPts pequeño



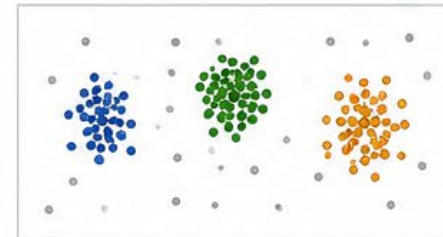
Muchos puntos se consideran parte del cluster (pocos outliers).

MinPts medio



Valor adecuado. Se obtienen clusters significativos y algo de ruido.

MinPts grande

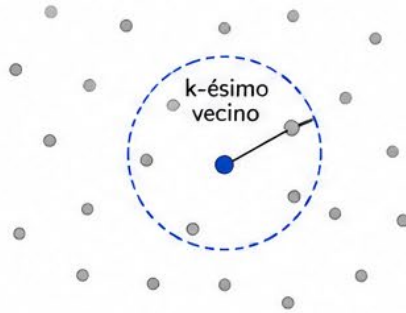


Muy estricto. Muchos puntos pasan a ser ruido, clusters más pequeños.

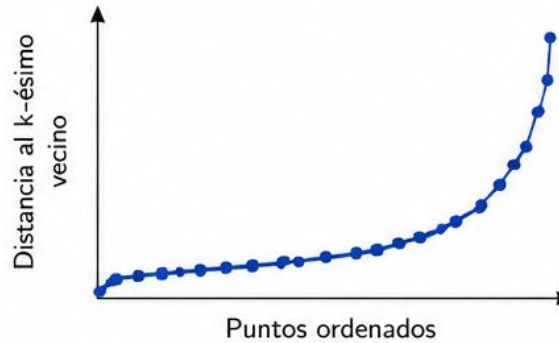
- Cluster 1
- Cluster 2
- Cluster 3
- Ruido (outliers)

Selección de ϵ con el gráfico k-distance

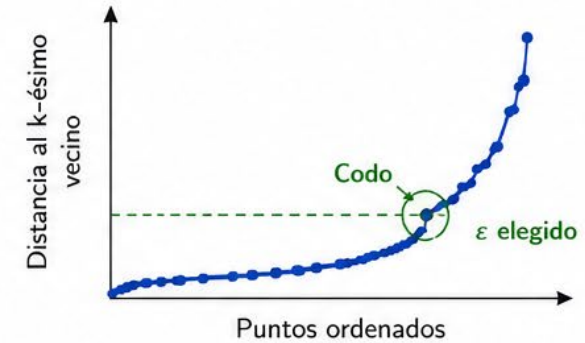
- 1 Calcular la distancia al k-ésimo vecino más cercano ($k = \text{MinPts}$)



- 2 Ordenar las distancias de menor a mayor y graficarlas



- 3 Elegir ϵ en el "codo" de la curva

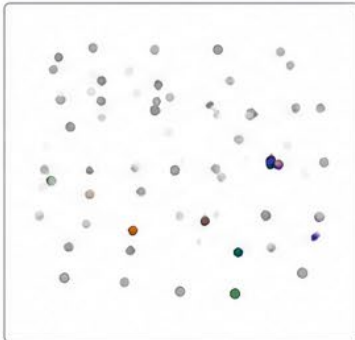


Cómo afecta ϵ en DBSCAN (ejemplo)

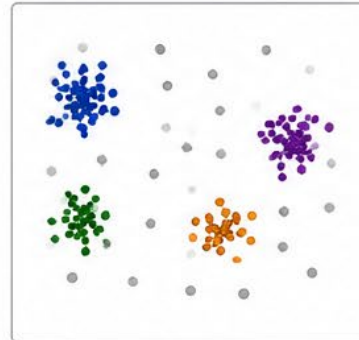
ϵ pequeño

ϵ adecuado

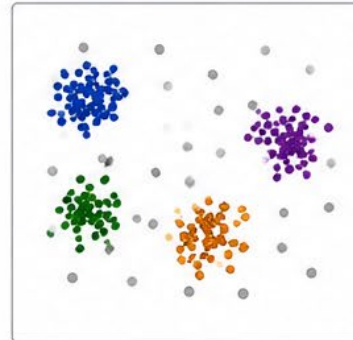
ϵ grande



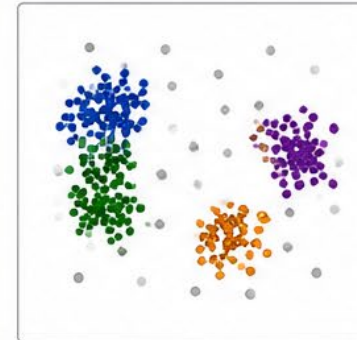
Muy pequeño



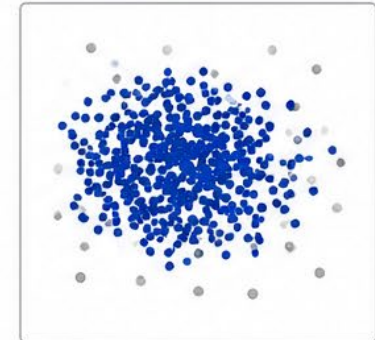
Pequeño



Adecuado



Grande



Muy grande

● Punto

● ● ● Clusters detectados

● Ruido (outliers)

REGLA PRÁCTICA PARA **MinPts**

$$\text{MinPts} \geq D + 1$$

Recomendación más robusta: $\text{MinPts} \approx 2 \times D$

donde D = número de dimensiones

EJEMPLOS

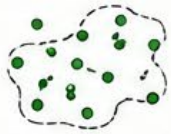
Dimensiones (D)	MinPts mínimo ($D + 1$)	MinPts recomendado ($2 \times D$)
2	3	4
3	4	6
5	6	10
10	11	20
20	21	40

DBSCAN: Ventajas y Desventajas

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) es un algoritmo de clustering basado en densidad.



VENTAJAS



1. Descubre clusters de forma arbitraria

Puede encontrar clusters de cualquier forma (no solo convexos).



2. Maneja ruido (outliers)

Identifica automáticamente los puntos de ruido sin necesidad de especificarlos previamente.



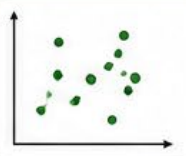
3. No requiere el número de clusters

A diferencia de otros algoritmos (como K-Means), no es necesario definir el número de clusters por adelantado.



4. Funciona bien con diferentes densidades

Puede encontrar clusters con distintas densidades en la misma ejecución.



5. Escalable

Tiene buena escalabilidad en conjuntos de datos grandes (con estructuras espaciales adecuadas como k-d tree, Ball tree, etc.).



DESVENTAJAS



MinPts

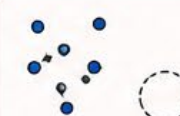
1. Sensible a la elección de parámetros

El resultado depende fuertemente de ϵ (epsilon) y MinPts. Elegir valores inadecuados puede generar mucho ruido o unir clusters distintos.



2. Dificultad con densidades muy variables

Si las diferencias de densidad entre clusters son muy grandes, puede fallar en detectar los clusters menos densos.



3. Dependiente de la escala

Es necesario escalar/normalizar las variables; de lo contrario, las distancias pueden estar dominadas por atributos con mayor magnitud.



4. Problemas en alta dimensionalidad

En espacios de alta dimensión, la noción de densidad se vuelve poco significativa (todas las distancias tienden a ser similares).



5. Puntos de borde ambiguos

Los puntos en las fronteras de los clusters pueden ser asignados de forma inconsistente dependiendo del orden de procesamiento.

DENSITY BASED SPATIAL CLUSTERING OF APPLICATIONS WITH NOISE

Rachit Vargas (MBA | Engineer)

rachit.vargas@ulead.ac.cr

(+506) 8474-3543